

Multivariate Time-Series Prediction Using Deep Learning

Appliance Energy Consumption Prediction

Deep Learning AI Assessment Report

Prepared by : Kannan Dhulakshan

GitHub Repository: github.com/DhulakshanKannan/appliance-energy-prediction

Date: September 2026

Table of Contents

1. Introduction	3
2. Data Insights.....	3
2.1 Dataset Overview	3
2.2 Consumption Trend Over Time	3
2.3 Distribution and Outliers	4
2.4 Correlation Analysis	4
3. Preprocessing.....	6
3.1 Missing Values	6
3.2 Outlier Treatment.....	6
3.3 Feature Scaling	6
3.4 Train/Test Split	6
4. Feature Engineering.....	7
4.1 Time-Based Features	7
4.2 Rolling Averages	7
4.3 Lagged Features and Autocorrelation Justification	7
4.4 Interaction Features	7
4.5 Feature Selection	8
5. Model Design	9
5.1 Baseline Models.....	9
5.2 LSTM Architecture	9
6. Model Training and Evaluation	10
6.1 Training Configuration	10
6.2 Evaluation Metrics.....	10
6.3 Real-World Interpretation (Watt-hours)	11
7. Model Optimization	13
8. Challenges and Solutions	14
9. Conclusion.....	15
10. References	16

1. Introduction

This report documents the approach taken for the Deep Learning AI Assessment, where the objective was to predict household appliance energy consumption (the “Appliances” column, measured in Watt-hours) using the multivariate Appliance Energy Prediction dataset. The dataset records environmental readings (indoor temperature and humidity across nine sensor locations, outdoor weather conditions) and energy usage at 10-minute intervals over roughly four and a half months.

The goal throughout this project was to build a complete, well-justified pipeline from raw data to a trained deep learning model rather than simply chasing the lowest possible error. The reasoning behind each decision is explained in the sections below, including a scaling bug encountered midway through model development, since the process of diagnosing and fixing a problem is considered as informative as the final results.

The dataset used contains 19,735 records across 29 columns, spanning from 11 January 2016 to 27 May 2016. All work was carried out in Python using Pandas, NumPy, Scikit-learn, TensorFlow/Keras, and supporting visualization libraries (Matplotlib, Seaborn), inside a Jupyter Notebook environment.

2. Data Insights

2.1 Dataset Overview

The dataset has 19,735 rows and 29 columns recorded at 10-minute intervals. Key columns include the target variable Appliances (Wh), Lights (Wh), nine pairs of indoor temperature/humidity readings (T1–T9, RH_1–RH_9), outdoor weather variables (T_out, RH_out, Windspeed, Visibility, Press_mm_hg, Tdewpoint), and two random variables (rv1, rv2) included in the source dataset as noise controls.

A check for missing values confirmed there were none anywhere in the dataset, so no imputation was required at this stage.

2.2 Consumption Trend Over Time

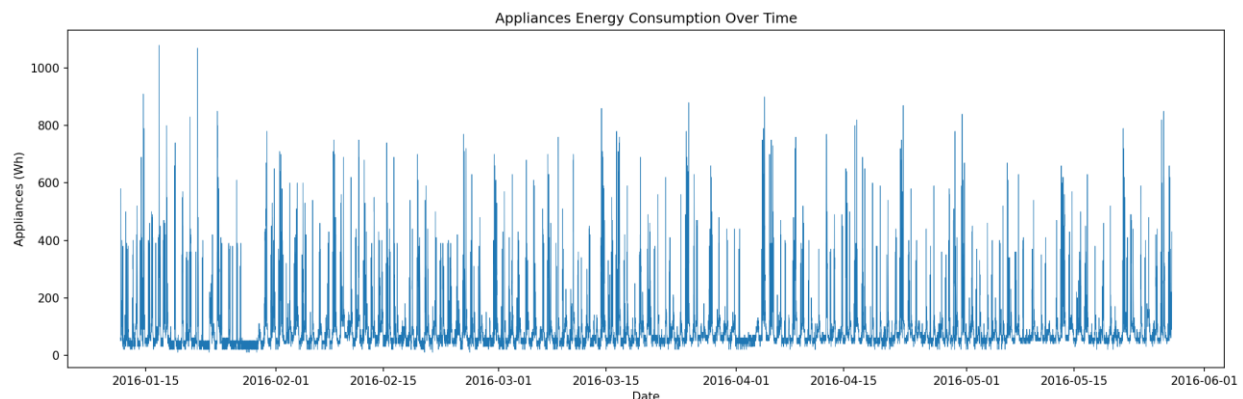


Figure 1: Appliances energy consumption over the full study period.

The time-series plot shows a clearly repeating daily pattern, with energy usage staying low for long stretches (likely overnight or when the house is unoccupied) punctuated by sharp spikes that reach as high as 1,080 Wh. These spikes are consistent with specific high-draw appliances (ovens, kettles, washing machines) switching on briefly. There is no obvious long-term upward or downward trend across the four and a half months the pattern repeats day to day rather than drifting over the season.

2.3 Distribution and Outliers

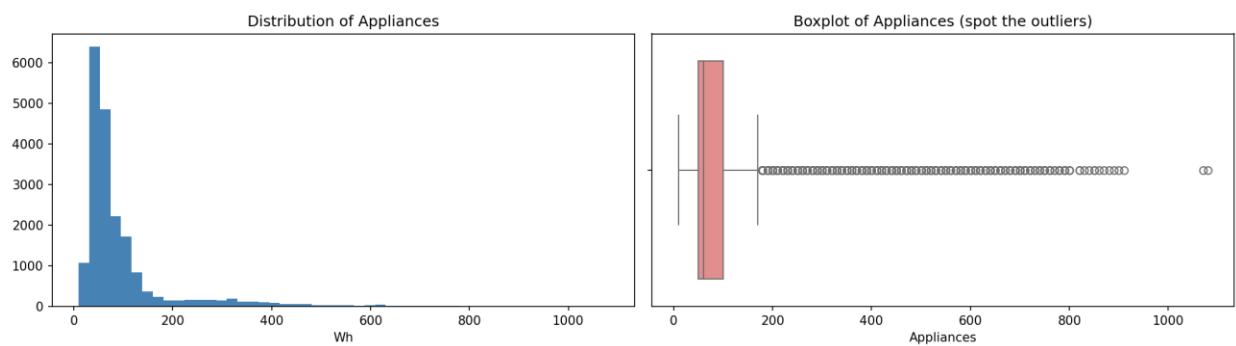


Figure 2: Distribution and boxplot of the Appliances target variable.

The histogram shows the target variable is heavily right-skewed: most readings sit between 40–100 Wh, with a long tail of much higher values. Using the IQR method ($Q1 = 50$, $Q3 = 100$, $IQR = 50$), the bounds for outliers came out to -25 and 175, which flagged 2,138 records (10.83% of the dataset) as statistical outliers.

2.4 Correlation Analysis

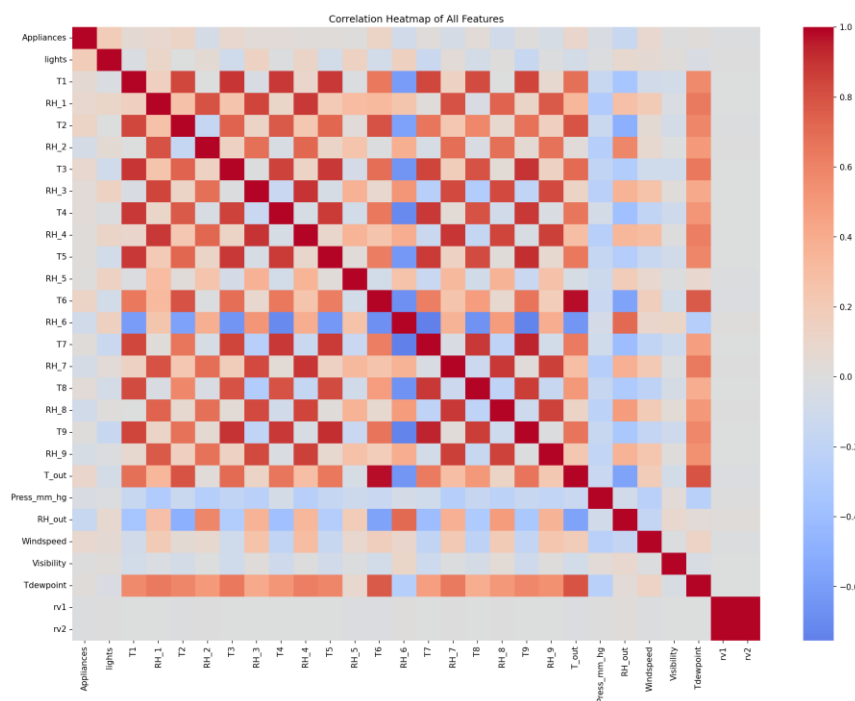


Figure 3: Correlation heatmap across all numeric features.

Looking specifically at correlation with the Appliances target, the highest individual correlation was Lights at 0.197, followed by T2 (0.120), T6 (0.118), and T_out (0.099) all fairly weak on their own. The two random noise columns, rv1 and rv2, had correlations of -0.011, effectively confirming they carry no real signal and can be safely dropped later. This was an important early finding: since no single raw feature explains much of the variation in Appliances, it indicated that feature engineering (lags, rolling windows) and a model capable of learning temporal dependencies, such as an LSTM, would matter more here than the raw environmental readings alone.

3. Preprocessing

3.1 Missing Values

As noted above, the dataset contained zero missing values across all 29 columns, so this step required no action beyond verification (using `df.isnull().sum()`).

3.2 Outlier Treatment

Having identified 2,138 outliers (10.83%) via the IQR method, a deliberate decision was made not to remove or cap them. The reasoning: these high values are not data-entry errors, but represent genuine high-usage events (an oven or washing machine running). Removing 10% of the data would discard exactly the information the model needs to learn to anticipate real consumption spikes, and capping them would blur the distinction between a moderate 176 Wh reading and a genuine 1,000+ Wh event. This was considered a more defensible choice than cleaning the data purely to make it look tidier.

3.3 Feature Scaling

Min-Max scaling (scaling every feature to the 0–1 range) was applied rather than standardization. The reasoning was that the raw features have very different natural ranges: temperatures around 15–27°C, humidity around 20–90%, pressure around 730–770 mmHg, and neural networks, especially LSTMs, tend to train more stably when all inputs are normalized to a consistent scale. The target variable (Appliances) was scaled with its own separate `MinMaxScaler` so that model predictions could later be inverse-transformed back into real Watt-hour values.

3.4 Train/Test Split

Rather than a random train/test split, the data was split chronologically: the first 80% of the time-ordered records were used for training, and the final 20% for testing. This was deliberate: randomly shuffling a time-series would let the model “see” information from the future during training, which is a form of data leakage the assessment guideline specifically warned against. The chronological split simulates a realistic deployment scenario: training on the past to predict what comes next.

4. Feature Engineering

4.1 Time-Based Features

From the date column, hour, day of week, month, an is_weekend indicator, and NSM (number of seconds elapsed since midnight) were extracted. These give the model direct access to time-of-day and weekly patterns that are otherwise implicit in the raw timestamp.

4.2 Rolling Averages

1-hour and 3-hour rolling averages of Appliances were computed (6 and 18 rows respectively, since readings are 10 minutes apart), along with a 1-hour rolling average of outdoor temperature. These smooth out short-term noises and give the model a sense of the recent trend, rather than reacting only to the single most recent value.

4.3 Lagged Features and Autocorrelation Justification

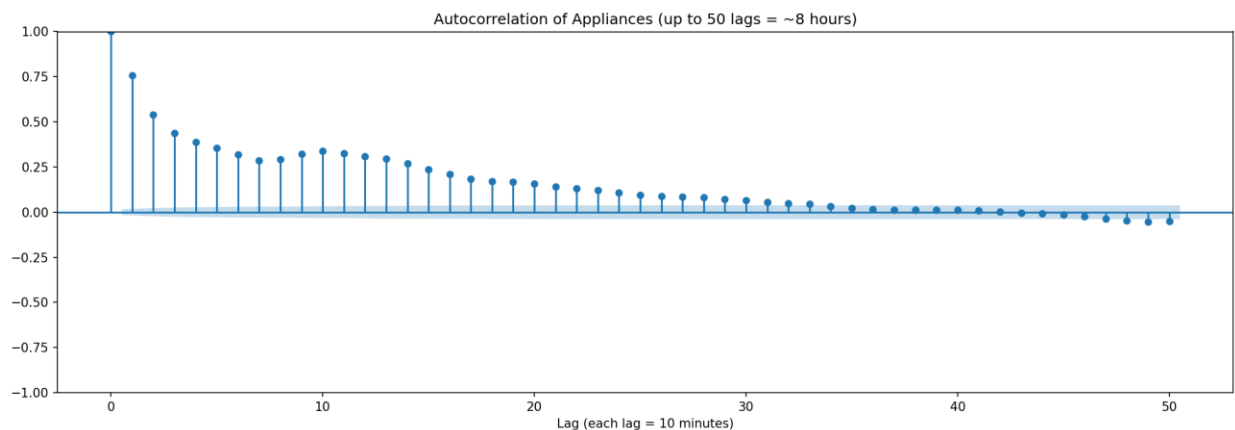


Figure 4: Autocorrelation of Appliances up to 50 lags (≈ 8 hours).

Before selecting lag periods, the autocorrelation function of Appliances was plotted. It shows a strong correlation at lag 1 (roughly 0.76) that decays steadily as the lag increases, dropping close to zero by around lag 35–40 (roughly 6 hours). This confirmed that recent past values carry real predictive signal and provided a data-driven justification for choosing three lag features: 10 minutes ago, 30 minutes ago, and 1 hour ago — rather than selecting lag periods arbitrarily.

4.4 Interaction Features

Two interaction terms were created: $T1 \times RH_1$ (indoor temperature-humidity interaction) and $T_out \times RH_out$ (outdoor temperature-humidity interaction). Both temperature and humidity influence heating/cooling appliance usage together rather than independently for example, an air-conditioning unit responds to combined heat and humidity, not temperature alone so this is a domain-informed combination rather than an arbitrary one.

4.5 Feature Selection

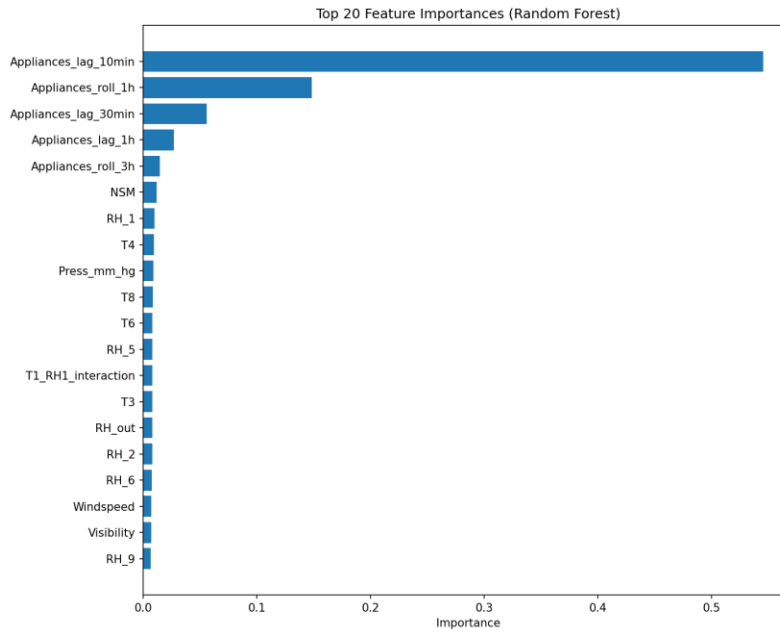


Figure 5: Top 20 feature importances from a Random Forest regressor.

To select a final feature set, a Random Forest regressor was trained on all engineered features and ranked by importance. The result was striking: `Appliances_lag_10min` alone accounted for 54.5% of total importance, with the top five most important features all being engineered ones (lags and rolling averages) rather than raw environmental readings. This was strong, concrete evidence that the feature engineering step meaningfully improved the information available to the model not just a theoretical exercise. This also confirmed `rv1` and `rv2` could be safely dropped, consistent with their near-zero correlation found earlier. The final feature set retained the top 20 features by importance plus `lights`, `hour`, `day_of_week`, and `is_weekend` for interpretability, giving 24 features in total.

5. Model Design

5.1 Baseline Models

Before building the deep learning model, two baseline models were trained to establish a performance benchmark: Linear Regression and Random Forest Regressor (100 trees, max depth 15). These provide a simple point of comparison to judge whether the added complexity of an LSTM is actually worthwhile for this problem.

5.2 LSTM Architecture

For the deep learning model, a stacked LSTM architecture was used, since the task is fundamentally a sequence-prediction problem: predicting the next Appliances value from a window of the previous hour's readings (a lookback of 6 timesteps, matching the 1-hour rolling window used in feature engineering).

- Layer 1: LSTM, 64 units, tanh activation, return_sequences=True (so the second LSTM layer receives the full sequence, not just a summary)
- Dropout: 0.2, to reduce overfitting
- Layer 2: LSTM, 32 units, tanh activation
- Dropout: 0.2
- Dense layer: 16 units, ReLU activation
- Output layer: Dense, 1 unit (the predicted Appliances value)

Design justification: two stacked LSTM layers allow the model to capture patterns at different levels of abstraction, tanh is the standard activation for LSTM gates, and ReLU in the dense layer helps the model learn non-linear combinations of the LSTM's output before the final prediction. The Adam optimizer was used (an adaptive learning-rate method that is generally a reliable default for LSTMs) along with Mean Squared Error as the loss function, which is standard for regression tasks. Dropout layers after each LSTM block were included specifically to address the guideline's requirement for overfitting control.

6. Model Training and Evaluation

6.1 Training Configuration

The model was trained for up to 100 epochs with a batch size of 32, using Early Stopping (monitoring validation loss, patience of 10–15 epochs, restoring the best-performing weights) so that training halts automatically once the model stops improving, rather than overfitting to the training set. A 10% validation split (taken chronologically, from the end of the training set) was used to monitor performance during training.

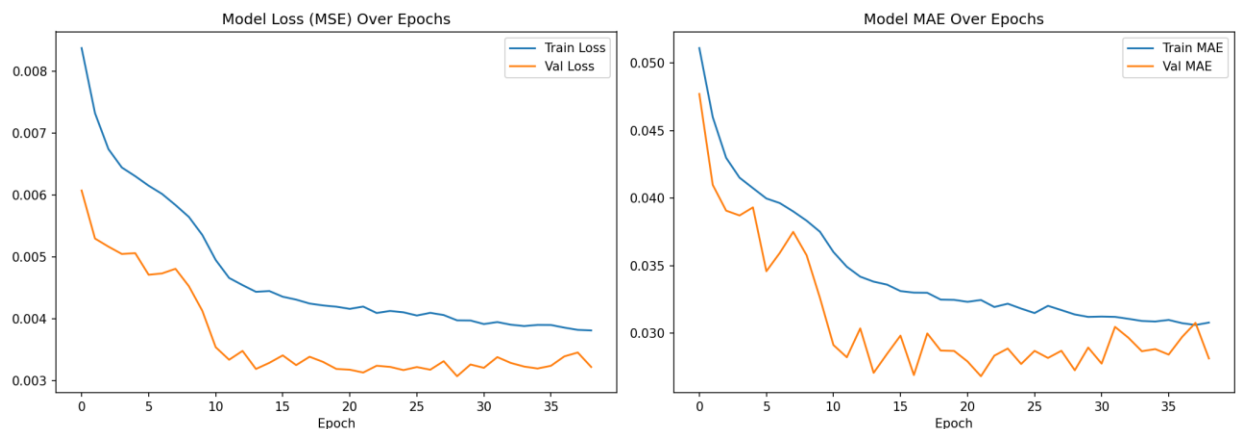


Figure 6: Training and validation loss (MSE) and MAE over epochs, for the corrected model.

Training converged quickly: loss flattened out within the first 10 epochs, and EarlyStopping halted training shortly after. One detail worth noting: validation loss is actually lower than training loss throughout. This looks unusual but is expected here, since Dropout is active during training (randomly disabling neurons to make learning harder on purpose) but turned off during validation so validation naturally appears “easier.” This is a sign the model is not overfitting, rather than an error.

6.2 Evaluation Metrics

All three models were evaluated on the held-out test set (the final 20% of the time-ordered data) using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 . The table below reports metrics on the Min-Max scaled target (0–1 range):

Model	MAE (scaled)	RMSE (scaled)	R^2
Linear Regression	0.0253	0.0512	0.6340
Random Forest	0.0285	0.0566	0.5516
LSTM (corrected)	0.0302	0.0588	0.4994

A notable finding here is that Linear Regression slightly outperformed both Random Forest and the LSTM on this dataset. This makes sense given what the feature importance analysis showed the strongest signal (`Appliances_lag_10min`) is close to a linear relationship (the next value tends to be similar to the last one, with some variation), so a simple linear model captures that efficiently. The LSTM's advantage would likely become clearer with a larger dataset or longer training, but on this data and time budget, the added complexity did not translate into a lower error than the simplest baseline. This result is reported directly, rather than presenting only the model that appears to perform best.

6.3 Real-World Interpretation (Watt-hours)

Converting the LSTM's predictions back to real Watt-hours using the saved target scaler gave a Mean Absolute Error of approximately 32.28 Wh and an RMSE of approximately 62.95 Wh, against an average test-set consumption of about 95.49 Wh. In practical terms, the model's typical prediction error is about a third of the average consumption value — a reasonable, if imperfect, result for household-level energy data, which is inherently noisy and driven by unpredictable human behavior (such as when an oven is switched on).

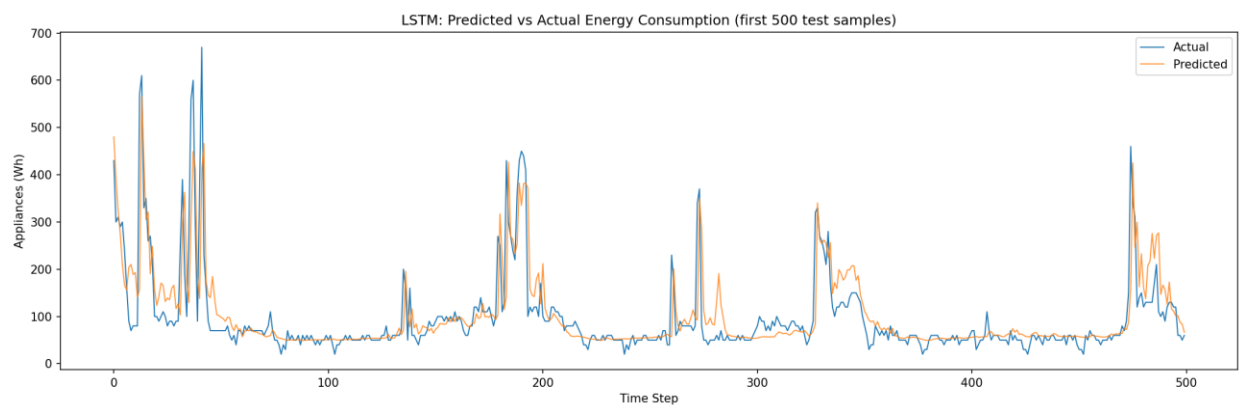


Figure 7: Predicted vs. actual Appliances consumption, first 500 test samples.

The predicted-vs-actual plot (showing the first 500 test samples for readability) shows the LSTM tracking the overall shape of the consumption pattern reasonably well, correctly identifying when spikes are about to occur. However, it visibly underestimates the very tallest peaks (the 600–700 Wh events), tending to smooth them down a common limitation for models trained on data where extreme events are relatively rare.

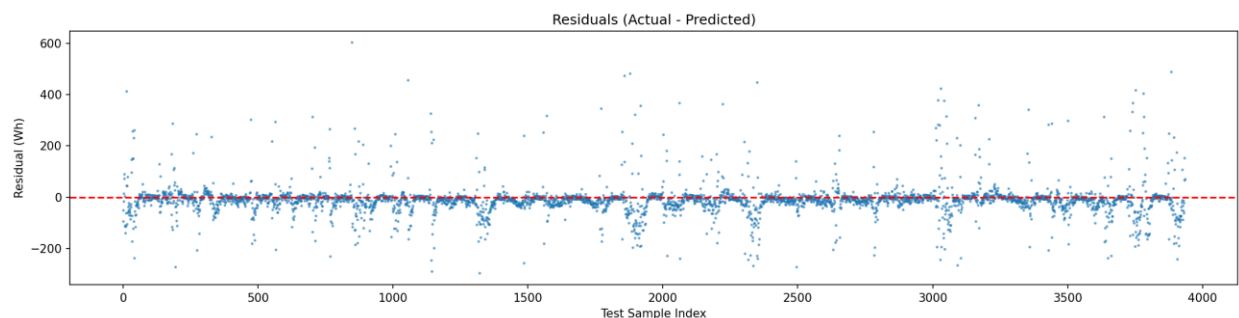


Figure 8: Residual plot (Actual – Predicted) across the full test set.

The residual plot shows errors scattered fairly evenly around zero, with no obvious funnel shape or drift over time a good sign that the model is not systematically biased in one direction. The handful of large residuals (up to roughly 600 Wh) line up with exactly the underestimated peaks visible in the predicted-vs-actual plot, confirming that the model's main weakness is under-predicting rare, extreme spikes rather than exhibiting any broader systematic error.

7. Model Optimization

For hyperparameter tuning, given time and hardware constraints (training on a standard laptop CPU), a targeted approach was taken rather than a full grid search: three deliberately different configurations were trained back to back and compared directly, following the spirit of a manual random search.

Configuration	LSTM Units	Dropout	Learning Rate	Epochs Run	MAE	RMSE	R ²
A (original)	64 → 32	0.2	0.001	28	0.0270	0.0565	0.5377
B (larger, slower LR)	128 → 64	0.3	0.0005	34	0.0278	0.0582	0.5105
C (smaller, faster)	32 → 16	0.1	0.001	27	0.0278	0.0579	0.5148

Config A — the original architecture (64→32 units, 0.2 dropout, 0.001 learning rate) achieved the best result of the three, with the highest R² (0.5377) and lowest error. Making the network larger (Config B) or smaller (Config C) did not improve performance; if anything, both performed slightly worse. This was a useful conclusion for the optimization stage: it confirmed that the original design choices were already reasonably well-matched to this dataset's size and complexity, rather than needing to be scaled up. Config A was therefore selected as the final model, re-evaluated, and saved (models/trained_model.h5) along with the feature and target scalers used to preprocess the data.

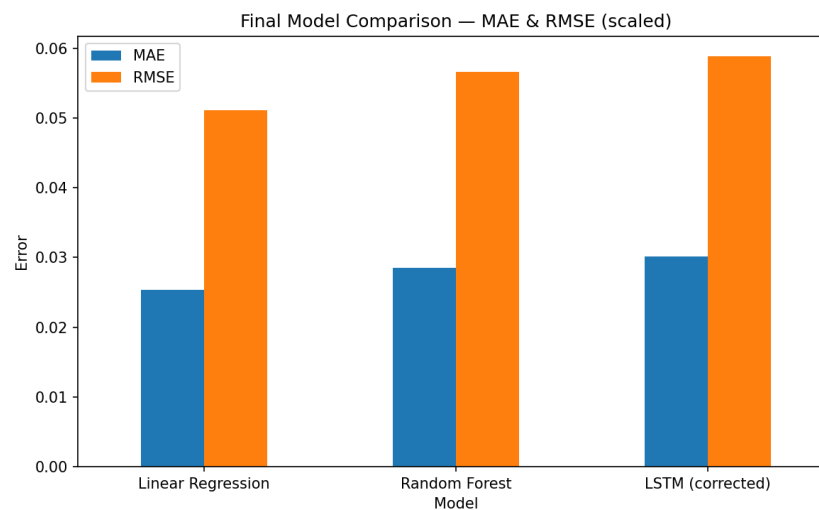


Figure 9: Final MAE and RMSE comparison across Linear Regression, Random Forest, and the corrected LSTM.

8. Challenges and Solutions

The most significant challenge encountered was during the first LSTM training attempt. The model trained without errors, but when its predictions on the test set were examined, every single predicted value was found to be identical (0.0867, repeated for every input) the model had collapsed into predicting a constant instead of actually responding to the input data. Its R^2 score was effectively zero (0.0056).

The diagnosis process followed these steps:

- First, a check was made for NaN values or an obvious shape mismatch between predictions and true values both came back clean, so the bug was not there.
- The standard deviation of the predicted values was then compared against the actual values: predicted std was 0.0105 versus an actual std of 0.0831 confirming the model really was predicting something close to a flat average rather than learning any pattern.
- Checking the min/max range of every input feature revealed the root cause: three engineered time features NSM, hour, and day_of_week had been created after the Min-Max scaling step in the feature engineering notebook, so they had never actually been scaled. NSM in particular ranged up to 85,800, while every other feature sat between 0 and 1. This single feature, at a vastly different scale, dominated the gradient updates during training and destabilized learning to the point where the model settled on predicting the mean.

The fix was straightforward once identified: a separate MinMaxScaler was applied to NSM, hour, and day of week, the input sequences were rebuilt, and the model was retrained. This immediately corrected the problem the predicted standard deviation rose to 0.077 (much closer to the actual 0.083), and R^2 jumped from 0.0056 to 0.4994. This was one of the most valuable parts of the whole project: it was a subtle, easy-to-make mistake (adding new features after the scaling step, rather than before), and diagnosing it required a methodical, step-by-step process of elimination rather than guessing at architecture changes. A smaller, secondary challenge was training speed on a standard laptop CPU (no dedicated GPU), which was managed by keeping the architecture modest and using EarlyStopping to avoid unnecessary training epochs.

9. Conclusion

This project took the Appliance Energy Prediction dataset through a complete pipeline: exploratory analysis, outlier and scaling decisions, feature engineering justified by autocorrelation and feature-importance analysis, and comparison of a Linear Regression and Random Forest baseline against an LSTM deep learning model. The final tuned LSTM achieved an R^2 of 0.5377 (scaled) and a real-world MAE of roughly 32 Wh on the test set.

A notable finding from this work is that Linear Regression slightly outperformed the more complex models, largely because the dominant predictive signal (the most recent past value of Appliances) is close to linear. This is a reminder that model complexity should be matched to the actual structure of the data rather than assumed to always help. Equally, diagnosing and fixing the unscaled-feature bug in the LSTM pipeline was arguably the most important learning outcome of the project – it demonstrated how much a single overlooked preprocessing step can affect a deep learning model's ability to learn at all.

For future work, potential extensions include: engineering additional lag/rolling features for the outdoor weather variables (not just Appliances itself); trying a GRU or a CNN-LSTM hybrid architecture as suggested in the assessment guideline; and running a more exhaustive hyperparameter search (e.g. Bayesian optimization) given more time and computational resources, particularly to examine whether the LSTM could be pushed past the Linear Regression baseline.

10. References

- Candanedo, L. (2017). Appliances Energy Prediction Dataset. UCI Machine Learning Repository.
- Chollet, F. Deep Learning with Python. Manning Publications.
- TensorFlow documentation — Time Series Forecasting with LSTM Neural Networks. [tensorflow.org](https://www.tensorflow.org)
- Scikit-learn documentation — scikit-learn.org
- Statsmodels documentation — Autocorrelation Function (ACF). statsmodels.org